PYTHON PROGRAMMING



21 Days Challenge

Day-21

Day-21

PYTHON PATTERN MATCHING AND DEBUGGING



Abdullah Al Zubaer



Instructor,
Artificial Intelligence & Data Science
Gobeshona Learning Academy.
B.Sc. in Internet of Things and Robotics Engineering
Gazipur Digital University, Gazipur, Bangladesh

Table Of Content

Regular Expressions

Debugging

Python Regex

- Regex = Regular Expressions (pattern matching)
- Used for input validation in authentication

Raw string

r"" → Use r to write a raw string so that backslashes are treated literally.

Pattern

- ^ → Start of string
- $ → End of string
- \w → [a-zA-Z0-9_]
- +@ → At least one character before and after @
- +\. → At least one character before and after .
- {2,10} → Minimum 2 characters

Pattern

- (?=.*[a-z]) → At least one lowercase letter
- (?=.*[A-Z]) → At least one uppercase letter
- (?=.*\d) → At least one digit
- (?=.*[@$!%*?&]) → At least one special character
- [A-Za-z\d@$!%*?&]{8,} → Allowed characters; minimum length 8

Regex Method

```
match()
Match only at start
re.match("Hi", "Hi John")

findall()
Find all matches, return list
re.findall("ai", "rain Spain")
```

Regex Method

```
search()
Find first match anywhere
re.search("dog", "I love dogs")


sub()
Replace pattern
re.sub("cat", "dog", "cat food")
```

Python Debugging

Debugging is the process of identifying, analyzing, and fixing errors or bugs in a program to ensure it runs correctly and as expected.

- Pylint
- pdb
- breakpoint()

Pylint in Python

Pylint is a static code analysis tool that checks for errors, enforces coding standards, and suggests improvements.

Pylint gives a score (/10) and shows style/syntax issues.

Pylint in Python

```
!pip install pylint

%%writefile my_script.py
a=1
b=2
print(a+b)

!pylint my_script.py
```

Breakpoints in Python

Breakpoints stop execution at a specific line so you can inspect values.

```
def divide(a, b):
    result = a / b
    breakpoint()  # execution stops here
    return result

divide(10, 2)
```

Breakpoints in Python

You'll enter an interactive debugger in the output cell.

Commands:

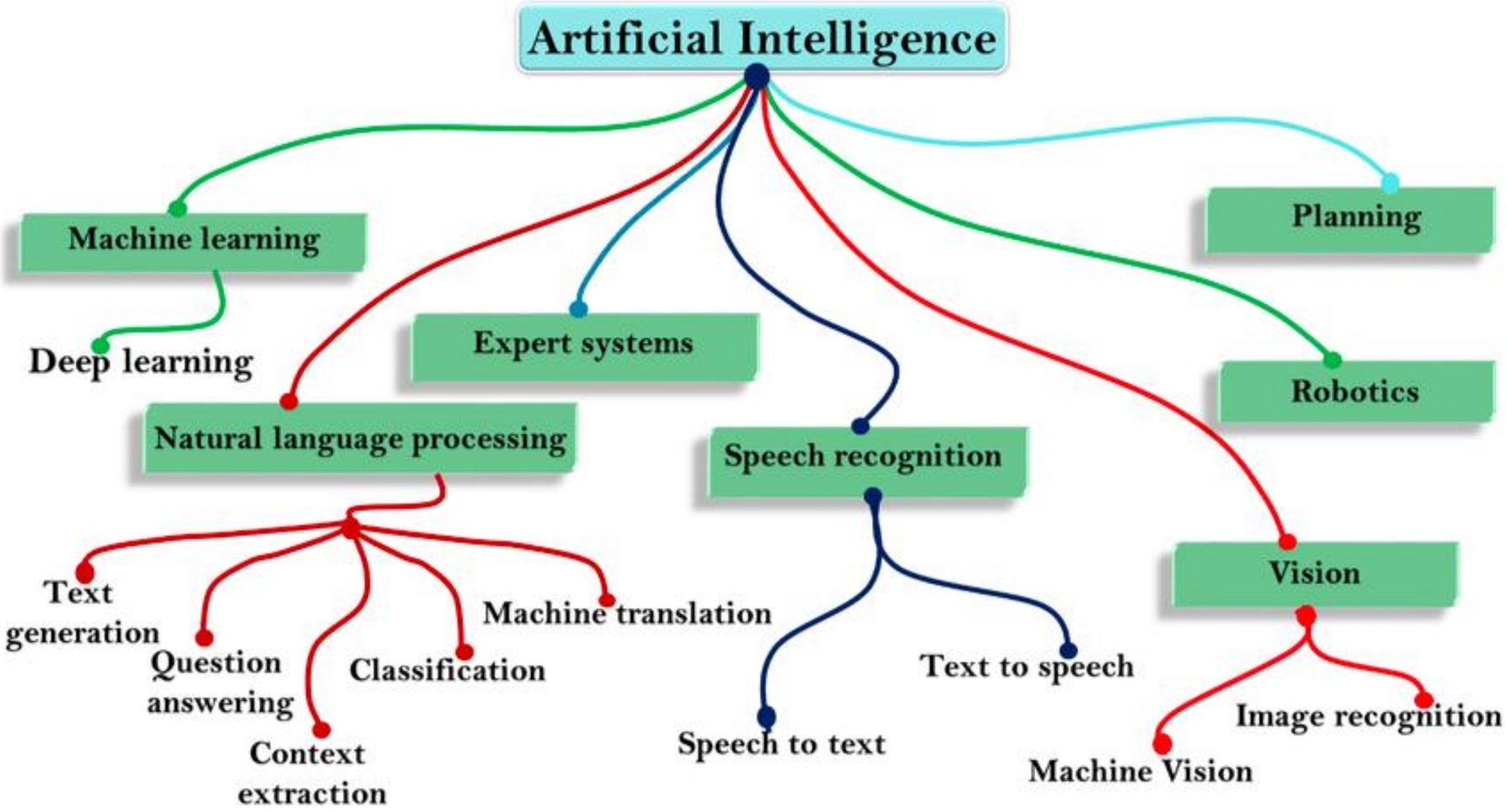
- n → next line
- c → continue
- p var → print variable
- q → quit debugger

pdb in Python

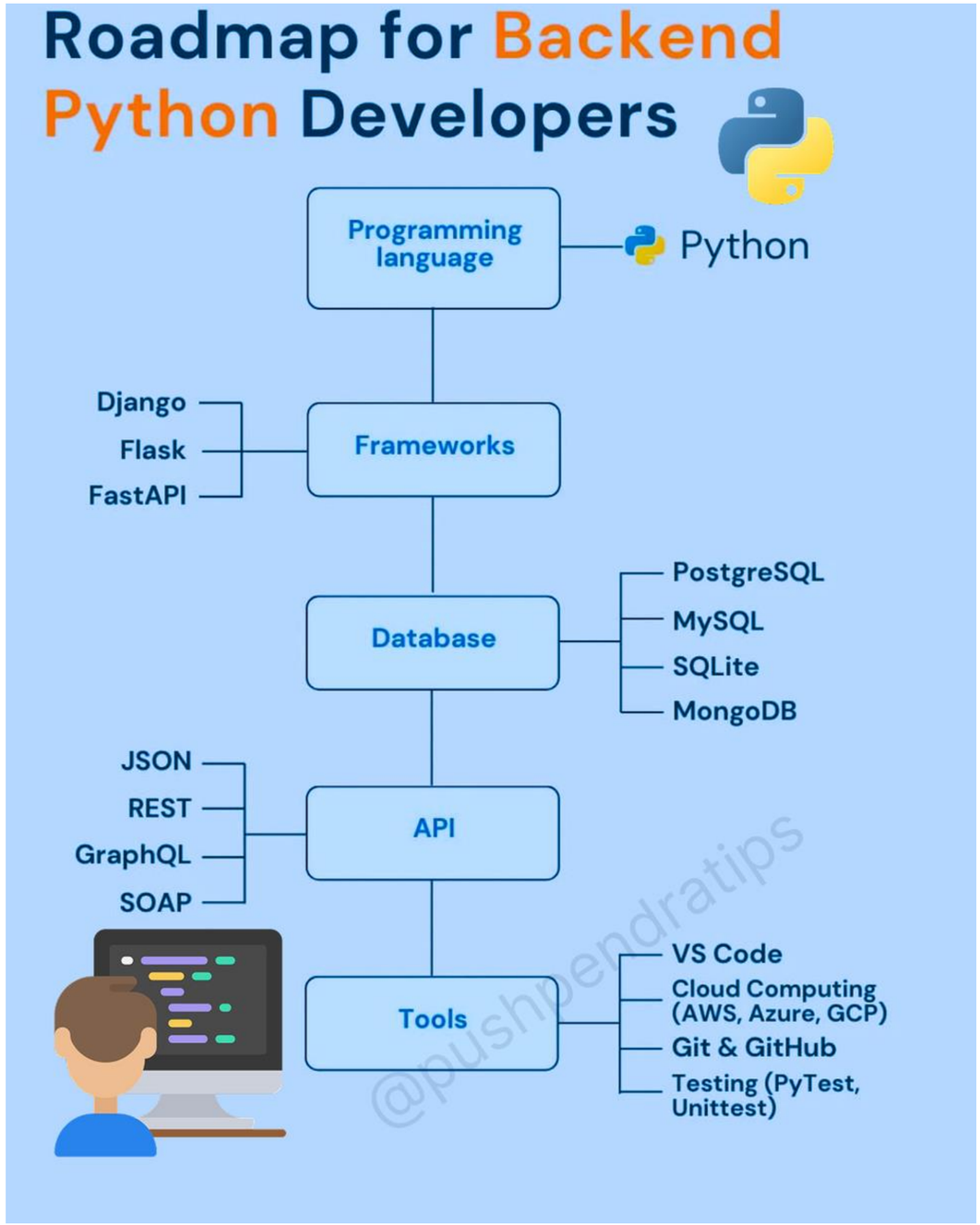
```
import pdb

def multiply(a, b):
    pdb.set_trace()  # debugger starts here
    return a * b

multiply(4, 5)
```


Artificial Intelligence
Machine learning
Deep learning
Expert systems
Natural language processing
Text generation
Question answering
Context extraction
Classification
Machine translation
Speech recognition
Speech to text
Text to speech
Planning
Robotics
Vision
Image recognition
Machine Vision

Roadmap for Backend Python Developers



QUESTION SESSION

THANK YOU

https://gobeshona.com.bd/